

SehatGuru: Updated Agentic Architecture

AI-Powered Nutrition System for Pakistani Cuisine

Core Components

1. Input Layer (What the LLM Receives)

- User health profile (BMI, weight, goals, activity level)
- Pakistani Dietary Guidelines (knowledge base)
- 1000+ Pakistani dishes database
- User's current food logs (daily intake)

2. Two Primary Functions

Function A: Nutritional Advice

- Uses RAG to retrieve relevant guidelines from knowledge base
- Contextualizes advice based on user's health profile
- Returns general dietary recommendations

Function B: Meal Plan Generation

- Retrieves dishes from 1000+ dish database
- Considers user's daily logs and remaining calorie/macro targets
- Generates culturally appropriate meal suggestions

Simplified Tech Stack

Component	Technology
Orchestration	LangGraph (for routing between advice/planning)
LLM	Gemini 1.5 Pro/Flash
Validation	Pydantic (structured outputs)
Database	Firebase, (RAG Retrieval ChromaDB)
Vector Search	pgvector (for knowledge base + dish retrieval)
Backend	FastAPI

The Validation Strategy

For Meal Plans: Python-Based Validation

Internal Validation (Built-in Logic)

```
class MealValidator:
    def validate_meal_plan(self, plan, user_context):
        # Check 1: Calorie bounds
        total_cals = sum([dish.calories for dish in plan])
        target = user_context.daily_target
        if not (target * 0.9 <= total_cals <= target * 1.1):
            return False, "Calorie target missed"

        # Check 2: Consider current logs
        consumed_today = user_context.consumed_calories
        remaining = target - consumed_today

        # Check 3: Macro balance (protein/carbs/fats)
        # Based on Pakistani dietary guidelines

        return True, "Valid plan"
```

For Nutritional Advice: Separate Validator Agent

Two-Step LLM Process

We use a separate LLM call to validate nutritional advice, creating a "Generator-Validator" pattern:

```
def generate_validated_advice(query, user_profile, kb_context):
    """
    Two LLM calls: one to generate, one to validate
    """
    MAX_ATTEMPTS = 3

    for attempt in range(MAX_ATTEMPTS):
        # Step 1: Generate advice (LLM Call #1)
        advice = generate_nutritional_advice(query, user_profile, kb_context)

        # Step 2: Validate advice (LLM Call #2)
        validation = validate_advice_with_llm(
            advice=advice,
            query=query,
            user_profile=user_profile,
            kb_context=kb_context
        )

        # Step 3: Check validation result
        if validation['approval_status'] == 'APPROVED':
            return {
                'advice': advice,
                'validation': validation,
                'attempts': attempt + 1
            }
    }
```

```

        # Step 4: If rejected, retry with feedback
        if validation['approval_status'] == 'NEEDS_REVISION':
            # Use validation feedback to improve next attempt
            continue

    # After 3 attempts, return error
    return {
        'advice': None,
        'error': 'Could not generate valid advice after 3 attempts',
        'last_validation': validation
    }

```

The Validator Agent

```

def validate_advice_with_llm(advice, query, user_profile, kb_context):
    """
    Separate LLM call to validate the generated advice
    """

```

```

    validation_prompt = f"""

```

You are a nutritional advice validator. Critically evaluate the following advice.

USER QUERY: {query}

USER PROFILE:

- Age: {user_profile.age}
- Weight: {user_profile.weight}kg
- BMI: {user_profile.bmi}
- Health Goal: {user_profile.goal}
- Medical Conditions: {user_profile.conditions}

PAKISTANI DIETARY GUIDELINES:

{kb_context}

ADVICE TO VALIDATE:

{advice}

VALIDATION CRITERIA:

1. SAFETY: Is the advice safe for this user's medical conditions? (Score 1-5)
2. ACCURACY: Is the advice factually correct per the guidelines? (Score 1-5)
3. PERSONALIZATION: Does it address the user's specific health goal? (Score 1-5)
4. CULTURAL RELEVANCE: Does it include Pakistani foods? (Score 1-5)
5. COMPLETENESS: Does it fully answer the question? (Score 1-5)

Respond in JSON format:

```

{{
    "safety_score": 1-5,
    "accuracy_score": 1-5,
    "personalization_score": 1-5,
    "cultural_score": 1-5,
    "completeness_score": 1-5,
    "overall_score": (average of above),
    "issues": ["list specific problems"],

```

```

        "corrections": ["list what should be changed"],
        "approval_status": "APPROVED" | "NEEDS_REVISION" | "REJECTED"
    }}
}

```

```

If overall_score >= 4.0, status = "APPROVED"
If overall_score 3.0-3.9, status = "NEEDS_REVISION"
If overall_score < 3.0, status = "REJECTED"
"""

```

```

validation_result = call_gemini(
    validation_prompt,
    temperature=0.2,
    response_format="json"
)

```

```

return validation_result

```

External Validation: Nutritionist Review Dataset

Building the Ground Truth

Week 1-2: Create Q&A pairs from knowledge base

```

test_dataset = [
    {
        'query': "I have diabetes, what should I eat?",
        'user_profile': {'has_diabetes': True, 'age': 45, 'bmi': 28},
        'expected_themes': ['low glycemic index', 'fiber', 'portion control'],
        'must_avoid': ['white rice', 'sugary foods']
    },
    {
        'query': "How much protein do I need for muscle gain?",
        'user_profile': {'weight': 70, 'goal': 'muscle_gain'},
        'expected_calculation': '1.6-2.2g per kg',
        'expected_foods': ['chicken', 'daal', 'eggs']
    }
    # ... 100 total questions
]

```

Week 2-3: Generate LLM responses

```

for test_case in test_dataset:
    response = generate_validated_advice(
        test_case['query'],
        test_case['user_profile'],
        knowledge_base
    )
    test_case['llm_response'] = response

```

Week 3-4: Nutritionist reviews using rubric

```

class NutritionistReviewRubric:
    CRITERIA = {
        'accuracy': 1-5,          # Factually correct?
        'safety': 1-5,           # Safe for user's profile?
        'completeness': 1-5,     # Addresses question fully?
        'practicality': 1-5,     # Realistic for Pakistani context?
        'clarity': 1-5           # Easy to understand?
    }

```

```
}  
MIN_PASSING_SCORE = 4.0 # Average across all criteria
```

Why This Approach?

For Nutritional Advice:

- Separate validator provides unbiased critique
- Can catch issues the generator missed
- Allows for iterative improvement (up to 3 attempts)
- More reliable validation for health-critical advice
- Nutritionist dataset provides final safety net

For Meal Plans:

- Python validation for math (LLMs are bad at arithmetic)
- Deterministic checks (calories, macros, portions)
- Fast validation (no extra LLM call needed)

Cost-Benefit Analysis

Aspect	Single Call	Separate Validator (Our Choice)
API Cost	1x per request	2x per request
Validation Quality	Good (self-check)	Better (unbiased critique)

Updated Workflow

For Meal Plan Generation:

1. Router → Identifies "meal plan request"
2. Context Injector → Pulls user's:
3. • Today's consumed meals
4. • Remaining calorie budget
5. • Health goals (weight loss/gain/maintain)
6. Dish Retriever (RAG) → Searches 200+ dishes
7. Plan Generator → Creates meal using:
8. • Pakistani dishes only
9. • Respects remaining calories
10. • Balances macros per guidelines
11. Validator Node → Python validation:
12. • Math check (calories sum correctly)
13. • Portion realism check
14. • Cultural appropriateness
15. Return to user OR loop back to step 4 if invalid

For Nutritional Advice:

1. Router → Identifies "advice request"
2. RAG → Retrieves from Pakistani dietary guidelines
3. Generator Agent (LLM Call #1) → Creates advice
4. Validator Agent (LLM Call #2) → Validates advice:
5. • Scores 5 criteria (safety, accuracy, personalization, cultural, completeness)
6. • Returns approval status
7. If APPROVED → Return to user
8. If NEEDS_REVISION → Loop back to step 3 (max 3 attempts)

9. If REJECTED or max attempts → Return error

Critical Implementation Notes

1. The "Current Logs" Integration

Agent state structure:

```
class AgentState(TypedDict):
    user_id: str
    daily_target_calories: int
    consumed_today: List[Dish] # Their logged meals
    remaining_calories: int    # Auto-calculated
    health_goal: str           # "lose_weight", etc.
```

2. Dish Database Structure

Each dish needs:

```
class PakistaniDish(BaseModel):
    dish_id: str
    name: str
    name_variants: List[str] # ["Karahi", "Karai", "Kadhai"]
    calories_per_serving: int
    protein_g: float
    carbs_g: float
    fats_g: float
    serving_size: str        # "1 plate", "2 roti"
    meal_type: str           # "breakfast", "lunch", "dinner"
```

3. The Nutritionist Validation Loop

- Week 1-2: Generate 100 Q&A pairs from knowledge base
- Week 2-3: LLM generates answers
- Week 3-4: Nutritionist reviews and corrects
- Week 4+: Use as test set for ongoing model evaluation

What You DON'T Need

- Complex multi-step re-planning loops for meal plans
- More than 3 validation attempts for advice
- Separate Python backup validation for advice

What You DO Need

- Accurate dish database (200+ dishes with verified nutrition info)
- Simple Python validator for meal plans
- Separate LLM validator agent for nutritional advice
- RAG for knowledge base (dietary guidelines)
- State management for daily logs
- Nutritionist-reviewed ground truth for continuous testing